

Einführung in die Theoretische Informatik

Nach Sipser, Kapitel 0 und 1

Sommersemester 2026

1 Kapitel 0: Einführung und mathematische Grundlagen

- 0.1 Automaten, Berechenbarkeit und Komplexität
- 0.2 Mathematische Grundbegriffe
- 0.3 Zeichenketten und Sprachen
- 0.4 Beweismethoden

2 Kapitel 1: Reguläre Sprachen

- 1.1 Endliche Automaten
- 1.2 Nichtdeterminismus
- 1.3 Reguläre Ausdrücke
- 1.4 Nichtregularität: Das Pumping-Lemma
- Zusammenfassung Kapitel 0 und 1

1 Kapitel 0: Einführung und mathematische Grundlagen

- 0.1 Automaten, Berechenbarkeit und Komplexität
- 0.2 Mathematische Grundbegriffe
- 0.3 Zeichenketten und Sprachen
- 0.4 Beweismethoden

2 Kapitel 1: Reguläre Sprachen

- 1.1 Endliche Automaten
- 1.2 Nichtdeterminismus
- 1.3 Reguläre Ausdrücke
- 1.4 Nichtregularität: Das Pumping-Lemma
- Zusammenfassung Kapitel 0 und 1

Drei zentrale Gebiete der Theoretischen Informatik

Die Theoretische Informatik beschäftigt sich mit den **Fähigkeiten und Grenzen** von Computern.

Drei eng verwandte Gebiete:

- 1 **Automatentheorie**: Welche mathematischen Modelle für Berechnung gibt es? Welche Probleme können sie lösen?
- 2 **Berechenbarkeitstheorie**: Was ist prinzipiell berechenbar – und was *nicht*?
- 3 **Komplexitätstheorie**: Welche Probleme sind berechenbar, welche sind leicht und welche sind *praktisch unlösbar* (zu schwer)?

Drei zentrale Gebiete der Theoretischen Informatik

Die Theoretische Informatik beschäftigt sich mit den **Fähigkeiten und Grenzen** von Computern.

Drei eng verwandte Gebiete:

- 1 **Automatentheorie**: Welche mathematischen Modelle für Berechnung gibt es? Welche Probleme können sie lösen?
- 2 **Berechenbarkeitstheorie**: Was ist prinzipiell berechenbar – und was *nicht*?
- 3 **Komplexitätstheorie**: Welche Probleme sind berechenbar, welche sind leicht und welche sind *praktisch unlösbar* (zu schwer)?

In dieser Vorlesung beginnen wir mit der **Automatentheorie** und den **regulären Sprachen** (Kapitel 1).

Berechenbarkeit:

- Manche Probleme sind *algorithmisch unlösbar* – kein Computer kann sie je lösen.
- Beispiel: Ist ein gegebenes mathematisches Theorem beweisbar?
- Für viele Probleme gibt es eine *prinzipielle* Grenze.

Berechenbarkeit:

- Manche Probleme sind *algorithmisch unlösbar* – kein Computer kann sie je lösen.
- Beispiel: Ist ein gegebenes mathematisches Theorem beweisbar?
- Für viele Probleme gibt es eine *prinzipielle* Grenze.

Komplexität:

- Manche Probleme *sind* lösbar, aber jeder bekannte Algorithmus braucht astronomisch viel Zeit.
- Beispiel: Optimale Tourenplanung (Traveling Salesman).
- Die **zentrale offene Frage**: $P \stackrel{?}{=} NP$.

Beide Gebiete brauchen ein **präzises mathematisches Fundament** – daher zunächst ein Überblick über die Grundlagen.

Definition

Eine **Menge** ist eine ungeordnete Zusammenfassung von Objekten (**Elemente**).

Schreibweisen:

- Aufzählung: $\{1, 2, 3\}$, $\{a, b, c\}$
- Beschreibung: $\{n \mid n \text{ ist gerade und } n > 0\} = \{2, 4, 6, \dots\}$
- $a \in S$: a ist Element von S ; $a \notin S$: a ist nicht Element von S

Definition

Eine **Menge** ist eine ungeordnete Zusammenfassung von Objekten (**Elemente**).

Schreibweisen:

- Aufzählung: $\{1, 2, 3\}$, $\{a, b, c\}$
- Beschreibung: $\{n \mid n \text{ ist gerade und } n > 0\} = \{2, 4, 6, \dots\}$
- $a \in S$: a ist Element von S ; $a \notin S$: a ist nicht Element von S

Wichtige Mengen:

- $\mathbb{N} = \{1, 2, 3, \dots\}$ (natürliche Zahlen; bei Sipser ohne 0)
- $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ (ganze Zahlen)
- $\emptyset = \{\}$ (leere Menge)

Teilmenge, Vereinigung, Durchschnitt

- **Teilmenge:** $A \subseteq B \iff$ jedes Element von A ist auch in B
- **Vereinigung:** $A \cup B = \{x \mid x \in A \text{ oder } x \in B\}$
- **Durchschnitt:** $A \cap B = \{x \mid x \in A \text{ und } x \in B\}$
- **Komplement:** $\bar{A} = \{x \mid x \notin A\}$ (bzgl. eines Universums)
- **Differenz:** $A \setminus B = \{x \mid x \in A \text{ und } x \notin B\}$

Teilmenge, Vereinigung, Durchschnitt

- **Teilmenge:** $A \subseteq B \iff$ jedes Element von A ist auch in B
- **Vereinigung:** $A \cup B = \{x \mid x \in A \text{ oder } x \in B\}$
- **Durchschnitt:** $A \cap B = \{x \mid x \in A \text{ und } x \in B\}$
- **Komplement:** $\bar{A} = \{x \mid x \notin A\}$ (bzgl. eines Universums)
- **Differenz:** $A \setminus B = \{x \mid x \in A \text{ und } x \notin B\}$

Potenzmenge

Die **Potenzmenge** $\mathcal{P}(A)$ ist die Menge aller Teilmengen von A .
Falls $|A| = n$, dann $|\mathcal{P}(A)| = 2^n$.

Beispiel: $A = \{0, 1\} \Rightarrow \mathcal{P}(A) = \{\emptyset, \{0\}, \{1\}, \{0, 1\}\}$.

Folge / Tupel

Eine **Folge** (oder ein **Tupel**) ist eine **geordnete** Liste von Objekten.

- 2-Tupel (Paar): (a, b)
- k -Tupel: $(a_1, a_2, \dots, a_k)$

Unterschied zu Mengen:

- Reihenfolge zählt: $(1, 2) \neq (2, 1)$
- Wiederholungen zählen: $(1, 1) \neq (1)$

Folge / Tupel

Eine **Folge** (oder ein **Tupel**) ist eine **geordnete** Liste von Objekten.

- 2-Tupel (Paar): (a, b)
- k -Tupel: $(a_1, a_2, \dots, a_k)$

Unterschied zu Mengen:

- Reihenfolge zählt: $(1, 2) \neq (2, 1)$
- Wiederholungen zählen: $(1, 1) \neq (1)$

Kartesisches Produkt

$$A \times B = \{(a, b) \mid a \in A, b \in B\}$$

$$\text{Allgemein: } A_1 \times A_2 \times \dots \times A_k = \{(a_1, \dots, a_k) \mid a_i \in A_i\}$$

$$\text{Beispiel: } \{a, b\} \times \{1, 2\} = \{(a, 1), (a, 2), (b, 1), (b, 2)\}.$$

Funktion

Eine **Funktion** $f: A \rightarrow B$ ordnet jedem Element $a \in A$ genau ein Element $f(a) \in B$ zu.

- A heißt **Definitionsbereich** (domain), B heißt **Wertebereich** (codomain).
- **Bild**: $\text{im}(f) = \{f(a) \mid a \in A\} \subseteq B$

Funktion

Eine **Funktion** $f: A \rightarrow B$ ordnet jedem Element $a \in A$ genau ein Element $f(a) \in B$ zu.

- A heißt **Definitionsbereich** (domain), B heißt **Wertebereich** (codomain).
- **Bild**: $\text{im}(f) = \{f(a) \mid a \in A\} \subseteq B$

Eigenschaften:

- **Injektiv** (eins-zu-eins): $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$
- **Surjektiv** (auf): $\text{im}(f) = B$
- **Bijektiv**: injektiv und surjektiv

Funktion

Eine **Funktion** $f: A \rightarrow B$ ordnet jedem Element $a \in A$ genau ein Element $f(a) \in B$ zu.

- A heißt **Definitionsbereich** (domain), B heißt **Wertebereich** (codomain).
- **Bild**: $\text{im}(f) = \{f(a) \mid a \in A\} \subseteq B$

Eigenschaften:

- **Injektiv** (eins-zu-eins): $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$
- **Surjektiv** (auf): $\text{im}(f) = B$
- **Bijektiv**: injektiv und surjektiv

Relation

Eine **k -stellige Relation** ist eine Teilmenge von $A_1 \times \dots \times A_k$.

Eine 2-stellige Relation $R \subseteq A \times A$ heißt **Äquivalenzrelation**, wenn R reflexiv, symmetrisch und transitiv ist.

Ungerichteter Graph

Ein **(ungerichteter) Graph** ist ein Paar $G = (V, E)$, wobei:

- V = endliche Menge von **Knoten**
- E = Menge von **Kanten** (2-elementige Teilmengen von V)

Gerichteter Graph

Ein **gerichteter Graph** ist ein Paar $G = (V, E)$ mit $E \subseteq V \times V$ (geordnete Paare).

Ungerichteter Graph

Ein **(ungerichteter) Graph** ist ein Paar $G = (V, E)$, wobei:

- V = endliche Menge von **Knoten**
- E = Menge von **Kanten** (2-elementige Teilmengen von V)

Gerichteter Graph

Ein **gerichteter Graph** ist ein Paar $G = (V, E)$ mit $E \subseteq V \times V$ (geordnete Paare).

Wichtige Begriffe:

- **Pfad**: Folge von Knoten, die durch Kanten verbunden sind
- **Zykel**: Pfad, der am Ausgangspunkt endet
- **Zusammenhängend**: zwischen je zwei Knoten existiert ein Pfad
- **Grad** eines Knotens: Anzahl der angrenzenden Kanten

Definition: Alphabet

Ein **Alphabet** Σ ist eine endliche, nichtleere Menge von Symbolen (**Zeichen**).

Beispiele: $\Sigma_1 = \{0, 1\}$, $\Sigma_2 = \{a, b, c, \dots, z\}$, $\Sigma_3 = \{0, 1, x, y, z\}$.

Definition: Alphabet

Ein **Alphabet** Σ ist eine endliche, nichtleere Menge von Symbolen (**Zeichen**).

Beispiele: $\Sigma_1 = \{0, 1\}$, $\Sigma_2 = \{a, b, c, \dots, z\}$, $\Sigma_3 = \{0, 1, x, y, z\}$.

Definition: Zeichenkette (Wort)

Eine **Zeichenkette** über Σ ist eine endliche Folge von Symbolen aus Σ .

- **Leere Zeichenkette:** ε (Länge 0)
- **Länge:** $|w|$ = Anzahl der Symbole
- **Umkehrung:** w^R (Spiegelung von w)
- **Konkatenation:** $x \cdot y$ (oder einfach xy): x gefolgt von y
- **Teilzeichenkette:** y ist Teilzeichenkette von x , falls $x = uyv$ für Zeichenketten u, v

Definition: Σ^*

Σ^* ist die **Menge aller Zeichenketten** über Σ (einschließlich ε).

Beispiel: $\Sigma = \{0, 1\} \Rightarrow \Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$.

Σ^* und formale Sprachen

Definition: Σ^*

Σ^* ist die **Menge aller Zeichenketten** über Σ (einschließlich ε).

Beispiel: $\Sigma = \{0, 1\} \Rightarrow \Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$.

Definition: Sprache

Eine **Sprache** ist eine Menge von Zeichenketten über einem Alphabet, also $L \subseteq \Sigma^*$.

Beispiele:

- $L_1 = \{w \in \{0, 1\}^* \mid w \text{ hat gleich viele Nullen und Einsen}\}$
- $L_2 = \{\text{alle syntaktisch korrekten Java-Programme}\}$
- Die Menge aller englischen Wörter über dem lateinischen Alphabet

In der Theoretischen Informatik studieren wir, welche *Maschinenmodelle* welche *Sprachen* erkennen können.

Mathematische Aussagen haben verschiedene Bezeichnungen:

- **Satz** (Theorem): wichtige, bewiesene Aussage
- **Lemma**: Hilfsaussage für einen größeren Beweis
- **Korollar**: Folgerung aus einem Satz

Mathematische Aussagen haben verschiedene Bezeichnungen:

- **Satz** (Theorem): wichtige, bewiesene Aussage
- **Lemma**: Hilfsaussage für einen größeren Beweis
- **Korollar**: Folgerung aus einem Satz

Beweise zeigen, dass eine Aussage wahr ist. In dieser Vorlesung begegnen uns vier wichtige Beweistechniken:

- 1 **Direkter Beweis**: Kette logischer Schlüsse von Voraussetzung zu Konklusion
- 2 **Beweis durch Konstruktion**: Existenz durch explizites Angeben eines Objekts
- 3 **Beweis durch Widerspruch**: Annahme der Negation $\leadsto$ logischer Widerspruch
- 4 **Beweis durch Induktion**: Basisfall + Induktionsschritt

Um zu zeigen, dass ein Objekt mit einer bestimmten Eigenschaft *existiert*, konstruiert man es explizit.

Beispiel

Behauptung: Für jede gerade Zahl $n \geq 2$ gibt es eine Menge mit genau $\frac{n}{2}$ Teilmengen der Größe 2.

Um zu zeigen, dass ein Objekt mit einer bestimmten Eigenschaft *existiert*, konstruiert man es explizit.

Beispiel

Behauptung: Für jede gerade Zahl $n \geq 2$ gibt es eine Menge mit genau $\frac{n}{2}$ Teilmengen der Größe 2.

Beweis: Betrachte $S = \{1, 2, \dots, n\}$. Konstruiere die Menge

$$\mathcal{F} = \{\{1, 2\}, \{3, 4\}, \dots, \{n-1, n\}\}.$$

$\mathcal{F}$ enthält genau $\frac{n}{2}$ Teilmengen der Größe 2. □

Konstruktive Beweise werden in der Automatentheorie ständig verwendet: z. B. “es gibt einen DFA, der die Sprache L erkennt” – wir geben den Automaten explizit an.

Beweis durch Widerspruch

Strategie: Nimm an, die Aussage sei *falsch*, und leite daraus einen logischen Widerspruch her.

Beispiel

Behauptung: $\sqrt{2}$ ist irrational.

Beweis durch Widerspruch

Strategie: Nimm an, die Aussage sei *falsch*, und leite daraus einen logischen Widerspruch her.

Beispiel

Behauptung: $\sqrt{2}$ ist irrational.

Beweis: Annahme: $\sqrt{2} = \frac{p}{q}$ mit $p, q \in \mathbb{Z}$, $q \neq 0$, $\gcd(p, q) = 1$.

Dann: $2 = \frac{p^2}{q^2}$, also $p^2 = 2q^2$.

$\Rightarrow p^2$ ist gerade $\Rightarrow p$ ist gerade (das Quadrat einer ungeraden Zahl ist ungerade), also $p = 2k$.

$\Rightarrow 4k^2 = 2q^2$, also $q^2 = 2k^2$, also q ist gerade.

Aber p und q beide gerade widerspricht $\gcd(p, q) = 1$. $\nexists$

$\Rightarrow \sqrt{2}$ ist irrational. □

Prinzip: Um eine Aussage $P(n)$ für alle $n \geq n_0$ zu beweisen:

- ① **Basisfall:** Zeige $P(n_0)$.
- ② **Induktionsschritt:** Zeige: $P(n) \Rightarrow P(n+1)$
(manchmal etwas stärkeres : $P(k) \text{ für alle } k \leq n \Rightarrow P(n+1)$).

Beweis durch Induktion

Prinzip: Um eine Aussage $P(n)$ für alle $n \geq n_0$ zu beweisen:

- ➊ **Basisfall:** Zeige $P(n_0)$.
- ➋ **Induktionsschritt:** Zeige: $P(n) \Rightarrow P(n+1)$
(manchmal etwas stärkeres : $P(k)$ für alle $k \leq n \Rightarrow P(n+1)$).

Beispiel

Behauptung: $\sum_{i=1}^n i = \frac{n \cdot (n+1)}{2}$ für alle $n \geq 1$.

Basisfall ($n = 1$): $1 = \frac{1 \cdot 2}{2} \checkmark$

Induktionsschritt ($n \rightarrow n+1$): Angenommen, die Aussage gilt für n . Dann:

$$\sum_{i=1}^{n+1} i = \left(\sum_{i=1}^n i \right) + (n+1) \stackrel{IV}{=} \frac{n \cdot (n+1)}{2} + (n+1) = \frac{(n+1) \cdot (n+2)}{2}. \quad \square$$

1 Kapitel 0: Einführung und mathematische Grundlagen

- 0.1 Automaten, Berechenbarkeit und Komplexität
- 0.2 Mathematische Grundbegriffe
- 0.3 Zeichenketten und Sprachen
- 0.4 Beweismethoden

2 Kapitel 1: Reguläre Sprachen

- 1.1 Endliche Automaten
- 1.2 Nichtdeterminismus
- 1.3 Reguläre Ausdrücke
- 1.4 Nichtregularität: Das Pumping-Lemma
- Zusammenfassung Kapitel 0 und 1

Endliche Automaten sind das einfachste Berechnungsmodell:

- Endlich viel Speicher (endlich viele Zustände)
- Eingabe wird zeichenweise von links nach rechts gelesen
- Am Ende: Akzeptieren oder Verwerfen

Endliche Automaten sind das einfachste Berechnungsmodell:

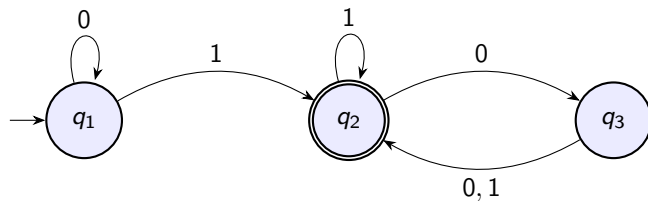
- Endlich viel Speicher (endlich viele Zustände)
- Eingabe wird zeichenweise von links nach rechts gelesen
- Am Ende: Akzeptieren oder Verwerfen

Anwendungen:

- Compilerbau (lexikalische Analyse)
- Textsuche und Mustererkennung
- Protokollverifikation
- Hardware-Steuerungen (Aufzüge, Ampeln, ...)

Beispiel: Ein einfacher Automat

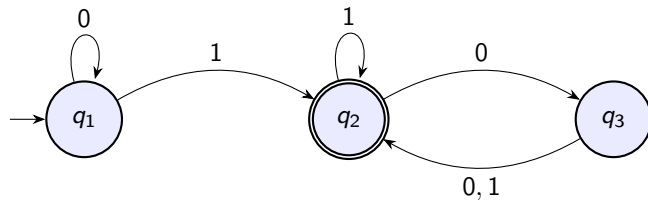
Automat M_1 mit Zustandsdiagramm:



- Startzustand: q_1 (Pfeil von außen)
- Akzeptierender Zustand: q_2 (Doppelkreis)
- M_1 akzeptiert z. B. 1, 01, 11, 0100 und verwirft 0, 10, ε .

Beispiel: Ein einfacher Automat

Automat M_1 mit Zustandsdiagramm:



- Startzustand: q_1 (Pfeil von außen)
- Akzeptierender Zustand: q_2 (Doppelkreis)
- M_1 akzeptiert z. B. 1, 01, 11, 0100 und verwirft 0, 10, ε .

Frage: Welche Sprache erkennt M_1 ?

$L(M_1) = \{w \in \{0, 1\}^* \mid \text{das letzte gelesene Zeichen einer 1 oder } \dots\}$ – wir werden das gleich formal präzisieren.

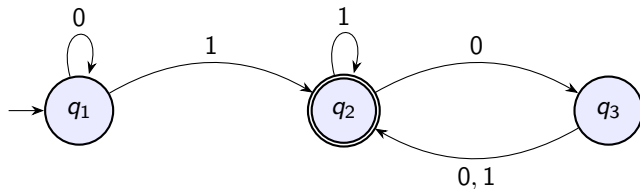
Definition

Ein **deterministischer endlicher Automat** (DFA) ist ein 5-Tupel

$$M = (Q, \Sigma, \delta, q_0, F)$$

- 1 Q ist eine endliche Menge von **Zuständen**.
- 2 Σ ist ein endliches **Eingabealphabet**.
- 3 $\delta: Q \times \Sigma \rightarrow Q$ ist die **Übergangsfunktion**.
- 4 $q_0 \in Q$ ist der **Startzustand**.
- 5 $F \subseteq Q$ ist die Menge der **akzeptierenden Zustände** (Endzustände).

Formale Beschreibung von M_1



$M_1 = (Q, \Sigma, \delta, q_1, F)$ mit:

- $Q = \{q_1, q_2, q_3\}$, $\Sigma = \{0, 1\}$, $F = \{q_2\}$
- Übergangsfunktion:

δ	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

Definition: Akzeptanz

Sei $M = (Q, \Sigma, \delta, q_0, F)$ ein DFA und $w = w_1 w_2 \cdots w_n$ eine Zeichenkette über Σ . M **akzeptiert** w , falls eine Folge von Zuständen $r_0, r_1, \dots, r_n \in Q$ existiert mit:

- ❶ $r_0 = q_0$ (Start im Startzustand)
- ❷ $r_{i+1} = \delta(r_i, w_{i+1})$ für $i = 0, \dots, n-1$ (Übergänge gemäß δ)
- ❸ $r_n \in F$ (Ende in einem akzeptierenden Zustand)

Berechnung eines DFA

Definition: Akzeptanz

Sei $M = (Q, \Sigma, \delta, q_0, F)$ ein DFA und $w = w_1 w_2 \cdots w_n$ eine Zeichenkette über Σ . M **akzeptiert** w , falls eine Folge von Zuständen $r_0, r_1, \dots, r_n \in Q$ existiert mit:

- ❶ $r_0 = q_0$ (Start im Startzustand)
- ❷ $r_{i+1} = \delta(r_i, w_{i+1})$ für $i = 0, \dots, n-1$ (Übergänge gemäß δ)
- ❸ $r_n \in F$ (Ende in einem akzeptierenden Zustand)

Definition: Erkannte Sprache

Die von M **erkannte Sprache** ist:

$$L(M) = \{w \in \Sigma^* \mid M \text{ akzeptiert } w\}$$

Definition: Reguläre Sprache

Eine Sprache heißt **regulär**, falls ein DFA existiert, der sie erkennt.

Entwurf endlicher Automaten

Strategie: Stell dir vor, du *bist* der Automat. Du liest die Eingabe Zeichen für Zeichen – was musst du dir merken?

Entwurf endlicher Automaten

Strategie: Stell dir vor, du *bist* der Automat. Du liest die Eingabe Zeichen für Zeichen – was musst du dir merken?

Beispiel: $L = \{w \in \{0,1\}^* \mid w \text{ enthält die Teilkette } 001\}$

Was muss der Automat “wissen”?

- Habe ich 001 schon gesehen? $\Rightarrow$ akzeptieren
- Wie viel vom Muster 001 habe ich bisher am aktuellen Suffix gesehen?

Entwurf endlicher Automaten

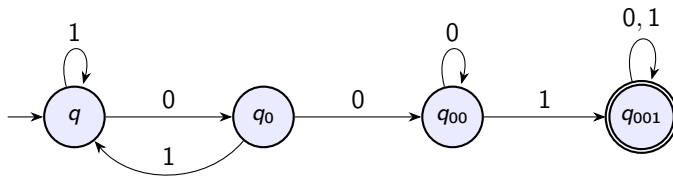
Strategie: Stell dir vor, du *bist* der Automat. Du liest die Eingabe Zeichen für Zeichen – was musst du dir merken?

Beispiel: $L = \{w \in \{0,1\}^* \mid w \text{ enthält die Teilkette } 001\}$

Was muss der Automat “wissen”?

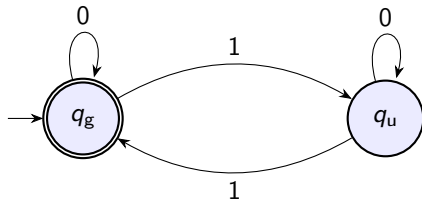
- Habe ich 001 schon gesehen? $\Rightarrow$ akzeptieren
- Wie viel vom Muster 001 habe ich bisher am aktuellen Suffix gesehen?

Zustände: “nichts”, “0 gesehen”, “00 gesehen”, “001 gefunden”



Beispiel: Gerade Anzahl von Einsen

$L = \{w \in \{0,1\}^* \mid \text{Anzahl der Einsen in } w \text{ ist gerade}\}$



- q_g : gerade Anzahl Einsen bisher (inkl. 0)
- q_u : ungerade Anzahl Einsen bisher
- ε hat 0 Einsen (gerade) $\Rightarrow$ Startzustand ist akzeptierend

Definition

Seien A und B Sprachen. Die **regulären Operationen** sind:

- **Vereinigung:** $A \cup B = \{w \mid w \in A \text{ oder } w \in B\}$
- **Konkatenation:** $A \circ B = \{xy \mid x \in A \text{ und } y \in B\}$
- **Stern:** $A^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ und } x_i \in A\}$

Definition

Seien A und B Sprachen. Die **regulären Operationen** sind:

- **Vereinigung:** $A \cup B = \{w \mid w \in A \text{ oder } w \in B\}$
- **Konkatenation:** $A \circ B = \{xy \mid x \in A \text{ und } y \in B\}$
- **Stern:** $A^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ und } x_i \in A\}$

Beispiele: $A = \{\text{gut, schlecht}\}$, $B = \{\text{Junge, Mädchen}\}$.

- $A \cup B = \{\text{gut, schlecht, Junge, Mädchen}\}$
- $A \circ B = \{\text{gutJunge, gutMädchen, schlechtJunge, schlechtMädchen}\}$
- $A^* = \{\varepsilon, \text{gut, schlecht, gutgut, gutschlecht, ...}\}$

Satz 1.25 (Sipser)

Die Klasse der regulären Sprachen ist abgeschlossen unter Vereinigung.
D. h. wenn A_1 und A_2 regulär sind, dann ist auch $A_1 \cup A_2$ regulär.

Satz 1.25 (Sipser)

Die Klasse der regulären Sprachen ist abgeschlossen unter Vereinigung.
D.h. wenn A_1 und A_2 regulär sind, dann ist auch $A_1 \cup A_2$ regulär.

Beweis (Produktkonstruktion): Seien $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ und $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ DFAs.
Konstruiere $M = (Q, \Sigma, \delta, q_0, F)$ mit:

- $Q = Q_1 \times Q_2$
- $q_0 = (q_1, q_2)$
- $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$
- $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$

M simuliert M_1 und M_2 **gleichzeitig** und akzeptiert, wenn *mindestens einer* akzeptiert. □

Bemerkung: Mit $F = F_1 \times F_2$ erhält man den Abschluss unter **Durchschnitt**.

Bemerkung: Formal muß die Korrektheit der Konstruktion gezeigt werden, folgt aber hier direkt aus der Konstruktion.

Beim DFA gibt es zu jedem Zustand und Symbol **genau einen** Folgezustand.

Nichtdeterminismus: Der Automat kann an manchen Stellen **mehrere Möglichkeiten** haben:

- Mehrere Folgezustände für dasselbe Symbol
- ε -Übergänge (Zustandswechsel ohne Eingabe zu lesen)
- Null Folgezustände (Berechnung “stirbt”)

Beim DFA gibt es zu jedem Zustand und Symbol **genau einen** Folgezustand.

Nichtdeterminismus: Der Automat kann an manchen Stellen **mehrere Möglichkeiten** haben:

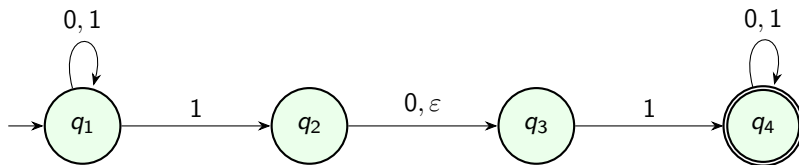
- Mehrere Folgezustände für dasselbe Symbol
- ϵ -Übergänge (Zustandswechsel ohne Eingabe zu lesen)
- Null Folgezustände (Berechnung “stirbt”)

Akzeptanz: Ein NFA akzeptiert, wenn *mindestens ein* möglicher Berechnungspfad in einen akzeptierenden Zustand führt.

Man kann sich den NFA als “magischen Rater” vorstellen: Er wählt immer den “richtigen” Pfad, falls einer existiert.

Bemerkung: Des bedeutet nicht da/ss ein NFA immer den richtigen Weg wählt!

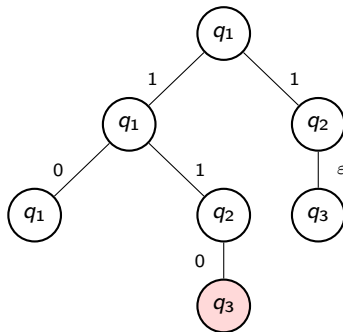
NFA – Beispiel N_1



- N_1 erkennt $L = \{w \in \{0,1\}^* \mid w \text{ enthält } 11 \text{ oder } 101 \text{ als Teilkette}\}$.
- In q_1 bei Eingabe 1: **zwei Möglichkeiten** – in q_1 bleiben oder nach q_2 gehen.
- ε -Übergang von q_2 nach q_3 : kein Zeichen wird gelesen.

Berechnungsbaum

Die Berechnung eines NFA lässt sich als **Baum** darstellen:



- Jeder *Blattknoten* ist ein mögliches Ende der Berechnung.
- NFA akzeptiert $\iff$ *mindestens ein Blatt* ist akzeptierend.

Definition

Ein **nichtdeterministischer endlicher Automat** (NFA) ist ein 5-Tupel

$$N = (Q, \Sigma, \delta, q_0, F)$$

- 1 Q ist eine endliche Zustandsmenge.
- 2 Σ ist das Eingabealphabet.
- 3 $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$ ist die Übergangsfunktion.
- 4 $q_0 \in Q$ ist der Startzustand.
- 5 $F \subseteq Q$ ist die Menge der akzeptierenden Zustände.

Wesentlicher Unterschied zum DFA:

$$\delta: Q \times \Sigma \rightarrow Q \quad (\text{DFA}) \quad \text{vs.} \quad \delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q) \quad (\text{NFA})$$

Definition

$N = (Q, \Sigma, \delta, q_0, F)$ akzeptiert w , falls sich w als $w = y_1 y_2 \cdots y_m$ schreiben lässt (mit $y_i \in \Sigma \cup \{\varepsilon\}$) und eine Folge von Zuständen $r_0, r_1, \dots, r_m \in Q$ existiert mit:

- ❶ $r_0 = q_0$
- ❷ $r_{i+1} \in \delta(r_i, y_{i+1})$ für $i = 0, \dots, m-1$
- ❸ $r_m \in F$

Beachte: $y_i \in \Sigma \cup \{\varepsilon\}$, d. h. ε -Übergänge sind erlaubt. Die Folge $y_1 \cdots y_m$ ergibt nach Entfernen aller ε das Wort w .

Satz 1.39 (Sipser)

Jeder NFA hat einen äquivalenten DFA.

Das bedeutet: Nichtdeterminismus erweitert die **Ausdruckskraft** endlicher Automaten *nicht* – nur die Bequemlichkeit.

Satz 1.39 (Sipser)

Jeder NFA hat einen äquivalenten DFA.

Das bedeutet: Nichtdeterminismus erweitert die **Ausdruckskraft** endlicher Automaten *nicht* – nur die Bequemlichkeit.

Korollar: Eine Sprache ist genau dann regulär, wenn ein NFA sie erkennt.

Beweis: durch die **Potenzmengenkonstruktion** (Teilmengenkonstruktion).

Beweis von Satz 1.39:

Gegeben: NFA $N = (Q, \Sigma, \delta, q_0, F)$.

Konstruiere DFA $M = (Q', \Sigma, \delta', q'_0, F')$ mit:

- $Q' = \mathcal{P}(Q)$ (jeder DFA-Zustand ist eine Menge von NFA-Zuständen)
- $q'_0 = E(\{q_0\})$ ($E(R)$: ε -Abschluss von R)
- Für $R \in Q'$ und $a \in \Sigma$:

$$\delta'(R, a) = E\left(\bigcup_{r \in R} \delta(r, a)\right)$$

- $F' = \{R \in Q' \mid R \cap F \neq \emptyset\}$

Beweis von Satz 1.39:

Gegeben: NFA $N = (Q, \Sigma, \delta, q_0, F)$.

Konstruiere DFA $M = (Q', \Sigma, \delta', q'_0, F')$ mit:

- $Q' = \mathcal{P}(Q)$ (jeder DFA-Zustand ist eine Menge von NFA-Zuständen)
- $q'_0 = E(\{q_0\})$ ($E(R)$: ε -Abschluss von R)
- Für $R \in Q'$ und $a \in \Sigma$:

$$\delta'(R, a) = E\left(\bigcup_{r \in R} \delta(r, a)\right)$$

- $F' = \{R \in Q' \mid R \cap F \neq \emptyset\}$

ε -Abschluss $E(R)$: Menge aller Zustände, die von R aus über beliebig viele ε -Übergänge erreichbar sind.

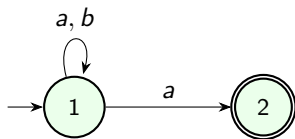
Korrektheit: M befindet sich nach Lesen von w im Zustand $R \iff R$ ist genau die Menge der Zustände, in denen N nach Lesen von w sein könnte.

Formaler Beweis: Induktion über die Länge von w .

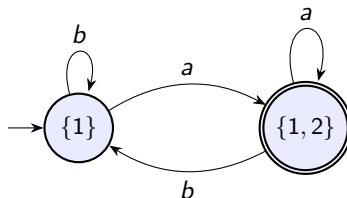
Potenzmengenkonstruktion – Beispiel

NFA N_2 : akzeptiert Wörter über $\{a, b\}$, die auf a enden.

NFA:



DFA (erreichbare Zustände):

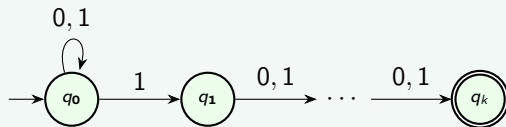


- Theoretisch $2^2 = 4$ DFA-Zustände, aber nur 2 sind erreichbar.
- In der Praxis: Nur erreichbare Zustände konstruieren!

Korollar: Exponentieller Blow-up

Beispiel

NFA über $\{0, 1\}$, der prüft, ob das k -letzte Zeichen eine 1 ist:



Dieser NFA hat $k + 1$ Zustände. Jeder äquivalente DFA benötigt **mindestens** 2^k Zustände!

⇒ Die Potenzmengenkonstruktion kann zu einem **exponentiellen Blow-up** führen – und das ist manchmal unvermeidbar.

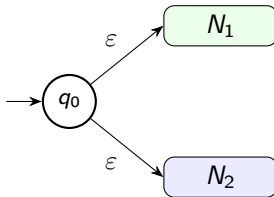
Abschluss unter den regulären Operationen

Satz 1.45 (Sipser): Abschluss unter Vereinigung

Wenn A_1, A_2 regulär sind, dann auch $A_1 \cup A_2$.

Beweis (mit NFA, einfacher als Produktkonstruktion):

Seien N_1, N_2 NFAs für A_1, A_2 . Konstruiere NFA N :



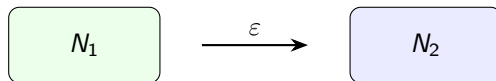
Neuer Startzustand q_0 mit ϵ -Übergängen zu den Startzuständen von N_1 und N_2 . □

Satz 1.47 (Sipser): Abschluss unter Konkatenation

Wenn A_1, A_2 regulär sind, dann auch $A_1 \circ A_2$.

Beweis:

Seien N_1, N_2 NFAs für A_1, A_2 . Konstruiere NFA N :



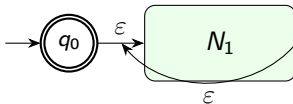
ϵ -Übergänge von jedem akzeptierenden Zustand von N_1 zum Startzustand von N_2 . Akzeptierende Zustände von N_1 sind in N *nicht mehr* akzeptierend. □

Satz 1.49 (Sipser): Abschluss unter Stern

Wenn A regulär ist, dann auch A^* .

Beweis:

Sei N_1 ein NFA für A . Konstruiere NFA N :



- Neuer Startzustand q_0 ist akzeptierend (für $\epsilon \in A^*$).
- ϵ -Übergang von q_0 zum Startzustand von N_1 .
- ϵ -Übergänge von akzeptierenden Zuständen von N_1 zurück zum Startzustand von N_1 (Wiederholung).

Definition 1.52 (Sipser)

R ist ein **regulärer Ausdruck** über Σ , falls R :

- 1 a für ein $a \in \Sigma$ (einzelnes Zeichen)
- 2 ε (leere Zeichenkette)
- 3 $\emptyset$ (leere Sprache)
- 4 $R_1 \cup R_2$, wobei R_1, R_2 reguläre Ausdrücke sind
- 5 $R_1 \circ R_2$, wobei R_1, R_2 reguläre Ausdrücke sind
- 6 R_1^* , wobei R_1 ein regulärer Ausdruck ist

Prioritäten: Stern $>$ Konkatenation $>$ Vereinigung

Beachte: ε und $\emptyset$ sind verschieden!

- $L(\varepsilon) = \{\varepsilon\}$ (Sprache mit einem Element: dem leeren Wort)
- $L(\emptyset) = \emptyset$ (Sprache ohne Elemente)

Reguläre Ausdrücke – Beispiele

Alphabet $\Sigma = \{0, 1\}$:

RegEx	Beschriebene Sprache
0^*10^*	Alle Wörter mit genau einer 1
$\Sigma^*1\Sigma^*$	Alle Wörter mit mindestens einer 1 (dabei: $\Sigma = 0 \cup 1$)
$\Sigma^*001\Sigma^*$	Alle Wörter mit Teilkette 001
$1^*(01^+)^*$	Alle Wörter ohne aufeinanderfolgende Nullen
$(\Sigma\Sigma)^*$	Alle Wörter gerader Länge

Reguläre Ausdrücke – Beispiele

Alphabet $\Sigma = \{0, 1\}$:

RegEx	Beschriebene Sprache
0^*10^*	Alle Wörter mit genau einer 1
$\Sigma^*1\Sigma^*$	Alle Wörter mit mindestens einer 1 (dabei: $\Sigma = 0 \cup 1$)
$\Sigma^*001\Sigma^*$	Alle Wörter mit Teilkette 001
$1^*(01^+)^*$	Alle Wörter ohne aufeinanderfolgende Nullen
$(\Sigma\Sigma)^*$	Alle Wörter gerader Länge

Kurzschreibweisen:

- Σ steht für $(0 \cup 1)$ (bei $\Sigma = \{0, 1\}$)
- R^+ steht für $R \circ R^*$ (mindestens einmal)
- R^k steht für $\underbrace{R \circ \dots \circ R}_{k\text{-mal}}$

Satz 1.54 (Sipser)

Eine Sprache ist genau dann regulär, wenn sie durch einen regulären Ausdruck beschrieben wird.

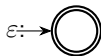
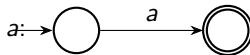
Zwei Richtungen:

- ① **RegEx** $\Rightarrow$ **NFA**: Lemma 1.55 (Konstruktion)
- ② **DFA** $\Rightarrow$ **RegEx**: Lemma 1.60 (Zustandseliminierung mit GNFA)

RegEx $\Rightarrow$ NFA (Lemma 1.55)

Beweis durch strukturelle Induktion über den Aufbau des regulären Ausdrucks.

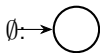
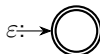
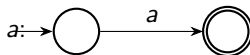
Basisfälle:



RegEx $\Rightarrow$ NFA (Lemma 1.55)

Beweis durch strukturelle Induktion über den Aufbau des regulären Ausdrucks.

Basisfälle:



Induktive Fälle:

- $R_1 \cup R_2$: neuer Start mit ε -Kanten zu NFAs von R_1 und R_2
- $R_1 \circ R_2$: Endzustände von R_1 via ε mit Start von R_2 verbinden
- R_1^* : neuer akzeptierender Start; Endzustände via ε zurück zum alten Start

Jeder Schritt produziert einen gültigen NFA $\Rightarrow$ Induktion.

DFA $\Rightarrow$ RegEx: Verallgemeinerte NFAs (GNFA)

Definition: GNFA

Ein **verallgemeinerter NFA** (GNFA) ist wie ein NFA, aber die Kanten sind mit **regulären Ausdrücken** statt einzelnen Symbolen beschriftet.

Spezielle Form:

- Genau ein Startzustand (mit Kanten zu allen anderen, keine eingehenden)
- Genau ein Endzustand (mit Kanten von allen anderen, keine ausgehenden)
- Zwischen je zwei Zuständen (außer Start/Ende) genau eine Kante (ggf. mit $\emptyset$)

DFA $\Rightarrow$ RegEx: Verallgemeinerte NFAs (GNFA)

Definition: GNFA

Ein **verallgemeinerter NFA** (GNFA) ist wie ein NFA, aber die Kanten sind mit **regulären Ausdrücken** statt einzelnen Symbolen beschriftet.

Spezielle Form:

- Genau ein Startzustand (mit Kanten zu allen anderen, keine eingehenden)
- Genau ein Endzustand (mit Kanten von allen anderen, keine ausgehenden)
- Zwischen je zwei Zuständen (außer Start/Ende) genau eine Kante (ggf. mit $\emptyset$)

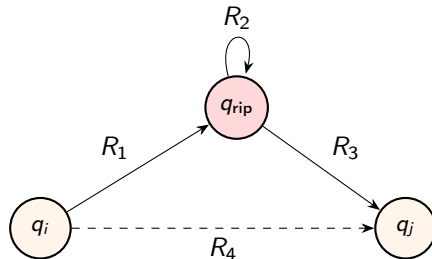
Zustandseliminierung:

- 1 Wandle DFA in einen GNFA mit $k + 2$ Zuständen um.
- 2 Wiederhole: Eliminiere einen Zustand und aktualisiere die Kanten-RegEx.
- 3 Am Ende: 2 Zustände (Start, Ende), eine Kante mit dem gesuchten RegEx.

GNFA: Zustandseliminierung

Eliminierung von Zustand q_{rip} :

Für alle Paare (q_i, q_j) mit $q_i \neq q_{\text{rip}} \neq q_j$:



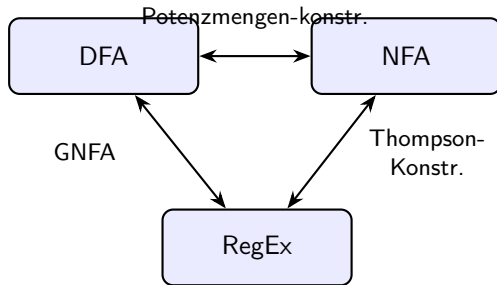
Neue Kante von q_i nach q_j :

$$R_{\text{neu}} = R_1 \circ R_2^* \circ R_3 \cup R_4$$

Korrektheit: Die neue Kante beschreibt genau alle Wörter, die den Automaten von q_i nach q_j führen (mit oder ohne Umweg über q_{rip}).



Zusammenfassung: Drei äquivalente Formalismen



Satz von Kleene

Für eine Sprache L sind äquivalent:

- ❶ L wird von einem DFA erkannt.
- ❷ L wird von einem NFA erkannt.
- ❸ L wird durch einen regulären Ausdruck beschrieben.

In all diesen Fällen heißt L **regulär**.

Nicht jede Sprache ist regulär

Frage: Gibt es Sprachen, die *nicht* regulär sind?

Intuitives Argument:

- Endliche Automaten haben nur *endlich* viel Speicher (endlich viele Zustände).
- Manche Sprachen erfordern es, sich eine *unbeschränkte* Menge an Information zu merken.

Nicht jede Sprache ist regulär

Frage: Gibt es Sprachen, die *nicht* regulär sind?

Intuitives Argument:

- Endliche Automaten haben nur *endlich* viel Speicher (endlich viele Zustände).
- Manche Sprachen erfordern es, sich eine *unbeschränkte* Menge an Information zu merken.

Beispiel: $B = \{0^n 1^n \mid n \geq 0\}$.

- Um zu prüfen, ob gleich viele Nullen wie Einsen vorliegen, muss man “zählen”.
- Ein endlicher Automat kann aber nur bis zu einer festen Grenze zählen (bestimmt durch die Zustandszahl).

⇒ Wir brauchen ein formales Werkzeug für solche Beweise.

Satz 1.70 (Pumping-Lemma)

Sei A eine reguläre Sprache. Dann existiert eine Zahl $p \geq 1$ (die **Pumping-Länge**), so dass jede Zeichenkette $s \in A$ mit $|s| \geq p$ in drei Teile $s = xyz$ zerlegt werden kann, die folgende Bedingungen erfüllen:

- 1 $\forall i \geq 0: xy^iz \in A$ (**Pumpen**)
- 2 $|y| > 0$ (y ist nicht leer)
- 3 $|xy| \leq p$ (x und y liegen im Präfix der Länge p)

Beweis:

Sei $M = (Q, \Sigma, \delta, q_1, F)$ ein DFA mit $L(M) = A$ und $p = |Q|$.

Sei $s = s_1 s_2 \cdots s_n \in A$ mit $n \geq p$. Sei $r_0, r_1, \dots, r_n$ die Zustandsfolge bei der Berechnung von M auf s (also $r_0 = q_1$ und $r_n \in F$).

Diese Folge hat $n + 1 \geq p + 1$ Elemente, aber nur $p = |Q|$ Zustände stehen zur Verfügung.

Pumping-Lemma – Beweis

Beweis:

Sei $M = (Q, \Sigma, \delta, q_1, F)$ ein DFA mit $L(M) = A$ und $p = |Q|$.

Sei $s = s_1 s_2 \cdots s_n \in A$ mit $n \geq p$. Sei $r_0, r_1, \dots, r_n$ die Zustandsfolge bei der Berechnung von M auf s (also $r_0 = q_1$ und $r_n \in F$).

Diese Folge hat $n + 1 \geq p + 1$ Elemente, aber nur $p = |Q|$ Zustände stehen zur Verfügung.

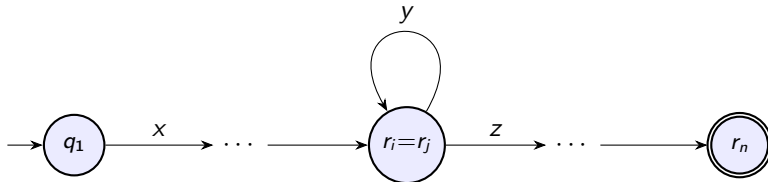
$\Rightarrow$ Nach dem **Schubfachprinzip** (Pigeonhole Principle) gibt es $i < j \leq p$ mit $r_i = r_j$.

Setze:

- $x = s_1 \cdots s_i$ (Pfad bis zur ersten Ankunft in r_i)
- $y = s_{i+1} \cdots s_j$ (Schleife von r_i zurück zu $r_i = r_j$)
- $z = s_{j+1} \cdots s_n$ (Rest bis zum Endzustand)

Dann: $|y| = j - i > 0 \checkmark$, $|xy| = j \leq p \checkmark$, und die Schleife y kann beliebig oft $(0, 1, 2, \dots \text{ mal})$ durchlaufen werden: $xy^i z \in A$ für alle $i \geq 0 \checkmark$. □

Visualisierung des Pumping-Lemmas



- x : Weg zum Beginn der Schleife
- y : die Schleife (mindestens ein Zustandswechsel)
- z : Weg von der Schleife zum akzeptierenden Zustand
- Schleife y kann $0, 1, 2, \dots$ Mal durchlaufen werden

Pumping-Lemma anwenden: Spielstruktur

Das Pumping-Lemma wird als **Widerspruchsbeweis** verwendet:

- 1 Annahme: L ist regulär.
- 2 Dann existiert eine Pumping-Länge p (unbekannt, vom “Gegner” gewählt).
- 3 **Wir** wählen ein geschicktes Wort $s \in L$ mit $|s| \geq p$.
- 4 **Der Gegner** wählt eine Zerlegung $s = xyz$ mit $|y| > 0$, $|xy| \leq p$.
- 5 **Wir** finden ein $i \geq 0$ mit $xy^iz \notin L$. ⚡

Pumping-Lemma anwenden: Spielstruktur

Das Pumping-Lemma wird als **Widerspruchsbeweis** verwendet:

- 1 Annahme: L ist regulär.
- 2 Dann existiert eine Pumping-Länge p (unbekannt, vom “Gegner” gewählt).
- 3 **Wir** wählen ein geschicktes Wort $s \in L$ mit $|s| \geq p$.
- 4 **Der Gegner** wählt eine Zerlegung $s = xyz$ mit $|y| > 0$, $|xy| \leq p$.
- 5 **Wir** finden ein $i \geq 0$ mit $xy^iz \notin L$. ⚡

Wichtig:

- Wir kennen p *nicht* – der Beweis muss für *jedes* p funktionieren.
- Wir kennen die Zerlegung *nicht* – der Beweis muss für *jede* gültige Zerlegung funktionieren.
- Nur die Wahl von s und i liegt bei uns.

Beispiel 1: $B = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

Beweis:

- ❶ Annahme: B ist regulär. Sei p die Pumping-Länge.
- ❷ Wähle $s = 0^p 1^p \in B$. ($|s| = 2p \geq p \checkmark$)
- ❸ Sei $s = xyz$ eine Zerlegung mit $|y| > 0$ und $|xy| \leq p$.
- ❹ Da $|xy| \leq p$, besteht xy nur aus Nullen. Also $y = 0^k$ für ein $k \geq 1$.
- ❺ Pumpe mit $i = 2$:

$$xy^2z = 0^{p+k}1^p \notin B \quad (\text{da } p+k \neq p).$$

- ❻ Widerspruch zum Pumping-Lemma!

$\Rightarrow B$ ist nicht regulär.



Beispiel 2: $C = \{w \mid w \text{ hat gleich viele 0en und 1en}\}$

Beweis:

- ❶ Annahme: C ist regulär. Sei p die Pumping-Länge.
- ❷ Wähle $s = 0^p 1^p \in C$. ($|s| = 2p \geq p \checkmark$)
- ❸ Sei $s = xyz$ mit $|y| > 0$, $|xy| \leq p$. Dann $y = 0^k$, $k \geq 1$.
- ❹ $xy^2z = 0^{p+k}1^p$ hat $p+k$ Nullen und p Einsen, also $xy^2z \notin C$. ⚡

$\Rightarrow C$ ist nicht regulär.



Beispiel 2: $C = \{w \mid w \text{ hat gleich viele 0en und 1en}\}$

Beweis:

- ❶ Annahme: C ist regulär. Sei p die Pumping-Länge.
- ❷ Wähle $s = 0^p 1^p \in C$. ($|s| = 2p \geq p \checkmark$)
- ❸ Sei $s = xyz$ mit $|y| > 0$, $|xy| \leq p$. Dann $y = 0^k$, $k \geq 1$.
- ❹ $xy^2z = 0^{p+k} 1^p$ hat $p+k$ Nullen und p Einsen, also $xy^2z \notin C$. ⚡

$\Rightarrow C$ ist nicht regulär. □

Bemerkung: Alternativ hätte man auch zeigen können, dass $B = C \cap 0^* 1^*$. Wäre C regulär, so auch B (Abschluss unter $\cap$). Da B nicht regulär ist, kann C nicht regulär sein.

Beispiel 3: $F = \{ww \mid w \in \{0,1\}^*\}$

Beweis:

- ❶ Annahme: F ist regulär. Sei p die Pumping-Länge.
- ❷ Wähle $s = 0^p 10^p 1 \in F$ (mit $w = 0^p 1$, also $|s| = 2p + 2 \geq p$).
- ❸ Sei $s = xyz$ mit $|y| > 0$, $|xy| \leq p$. Dann $y = 0^k$, $k \geq 1$, und y liegt komplett in der ersten Hälfte der Nullen.
- ❹ $xy^0z = xz = 0^{p-k} 10^p 1$.
- ❺ Damit $xz \in F$ müsste $xz = ww$ gelten. Aber xz hat $p - k$ Nullen vor der ersten 1 und p Nullen vor der zweiten 1. Da $k \geq 1$, kann xz nicht die Form ww haben. ⚡

$\Rightarrow F$ ist nicht regulär.



Beispiel 4: $D = \{1^{n^2} \mid n \geq 0\}$

Beweis:

- ➊ Annahme: D ist regulär. Sei p die Pumping-Länge.
- ➋ Wähle $s = 1^{p^2} \in D$. ($|s| = p^2 \geq p$ ✓)
- ➌ Sei $s = xyz$ mit $|y| > 0$, $|xy| \leq p$.
- ➍ Setze $k = |y|$. Dann $1 \leq k \leq p$.
- ➎ $|xy^2z| = p^2 + k$.
- ➏ Es gilt $p^2 < p^2 + k \leq p^2 + p < p^2 + 2p + 1 = (p+1)^2$.
- ➐ Also liegt $|xy^2z|$ **echt zwischen** zwei aufeinanderfolgenden Quadratzahlen.
- ➑ $\Rightarrow xy^2z \notin D$. ⚡

$\Rightarrow D$ ist nicht regulär.



Pumping-Lemma: Vorsicht!

Wichtige Bemerkungen

- Das PL ist eine **notwendige Bedingung** für Regularität, aber **nicht hinreichend**.
- Es gibt nichtreguläre Sprachen, die das PL erfüllen!
- Das PL kann **nur** Nicht-Regularität beweisen, *nie* Regularität.

Pumping-Lemma: Vorsicht!

Wichtige Bemerkungen

- Das PL ist eine **notwendige Bedingung** für Regularität, aber **nicht hinreichend**.
- Es gibt nichtreguläre Sprachen, die das PL erfüllen!
- Das PL kann **nur** Nicht-Regularität beweisen, *nie* Regularität.

Weitere Methoden zum Nachweis von Nicht-Regularität:

- **Abschlusseigenschaften**: Zeige, dass eine Operation regulärer Sprachen auf L eine bekannt nichtreguläre Sprache ergibt.
- **Myhill-Nerode-Satz**: Über die Anzahl der Äquivalenzklassen der Nerode-Relation (wird in manchen Vorlesungen gesondert behandelt).

Kapitel 0:

- Grundbegriffe: Mengen, Folgen, Funktionen, Relationen, Graphen
- Zeichenketten und formale Sprachen
- Beweismethoden: Konstruktion, Widerspruch, Induktion

Kapitel 0:

- Grundbegriffe: Mengen, Folgen, Funktionen, Relationen, Graphen
- Zeichenketten und formale Sprachen
- Beweismethoden: Konstruktion, Widerspruch, Induktion

Kapitel 1:

- **1.1:** DFA – Definition, Entwurf, reguläre Operationen, Abschluss unter $\cup$ (Produktkonstruktion)
- **1.2:** NFA – Definition, Äquivalenz zu DFA (Potenzmengenkonstruktion), Abschluss unter $\cup, \circ, *$
- **1.3:** Reguläre Ausdrücke – Definition, Äquivalenz zu endlichen Automaten (Thompson-Konstruktion, GNFA)
- **1.4:** Pumping-Lemma – Beweis und Anwendungen zum Nachweis nichtregullärer Sprachen

Die drei Säulen

- ➊ **Äquivalenz:** $\text{DFA} \equiv \text{NFA} \equiv \text{Regex}$ (**Satz von Kleene**)
- ➋ **Abschluss:** Reguläre Sprachen sind abgeschlossen unter $\cup, \cap, \bar{\cdot}, \circ, *$
- ➌ **Grenzen:** Das Pumping-Lemma zeigt, dass es nichtreguläre Sprachen gibt (z. B. $\{0^n 1^n\}$, $\{ww\}$, $\{1^{n^2}\}$)

Ausblick – Kapitel 2: Kontextfreie Sprachen und Kellerautomaten – ein stärkeres Modell, das z. B. $\{0^n 1^n\}$ erkennen kann.

Vielen Dank!

Fragen?